

Language and seq2seq Modeling

Language Models

Language Models

- Language Models estimate the probability of a single token
- It is more convenient to estimate the *conditional* probability:

$$P(y_1, y_2, \dots, y_n) = P(y_1) \cdot P(y_2|y_1) \cdot P(y_3|y_1, y_2) \cdot \dots \cdot P(y_n|y_1, \dots, y_{n-1}) = \prod_{t=1}^n P(y_t|y_{<t})$$

We need to:

1. Find a way to compute conditional probabilities
2. Use them to generate new tokens in *autoregressive* way

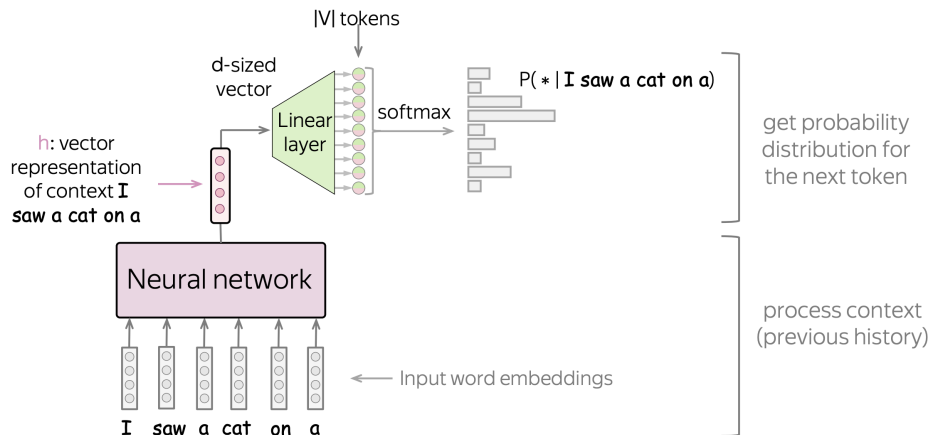
$$x_i = f(x_{i-1}, x_{i-2}, \dots)$$

Autoregressive LMs

How to compute conditional probabilities?

- Use any classifier model you know with $n_{classes} = vocab_size$
- Take a large corpus of texts
- Train a model using *Cross-entropy loss*

$$-\sum_{i=1}^{|V|} p_i^* \cdot \log P(y_t = i|x) \rightarrow \min (p_k^* = 1, p_i^* = 0, i \neq k)$$



Sequence to sequence models

Seq2seq models

Need to get one sequence from another (e.g. machine translation)

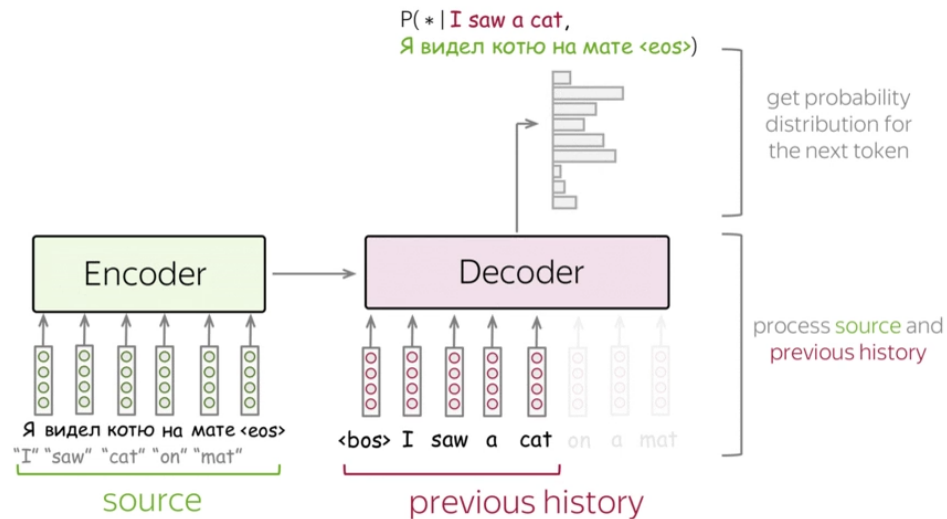
$$y^* = \arg \max_y p(y | x)$$

How to model?

Use **Encoder-Decoder** Framework:

- **Encoder** – projects source text x into *latent representation*
- **Decoder** – uses source representation from encoder to generate new sequence y
 - In particular, *conditional language model*

$$P(y_1, y_2, \dots, y_n | x) = \sum_{t=1}^n p(y_t | y_{<t}, x)$$



Attention mechanism

Attention (Bahdanau, 2014)

Motivation: problem with *fixed encoder representation*

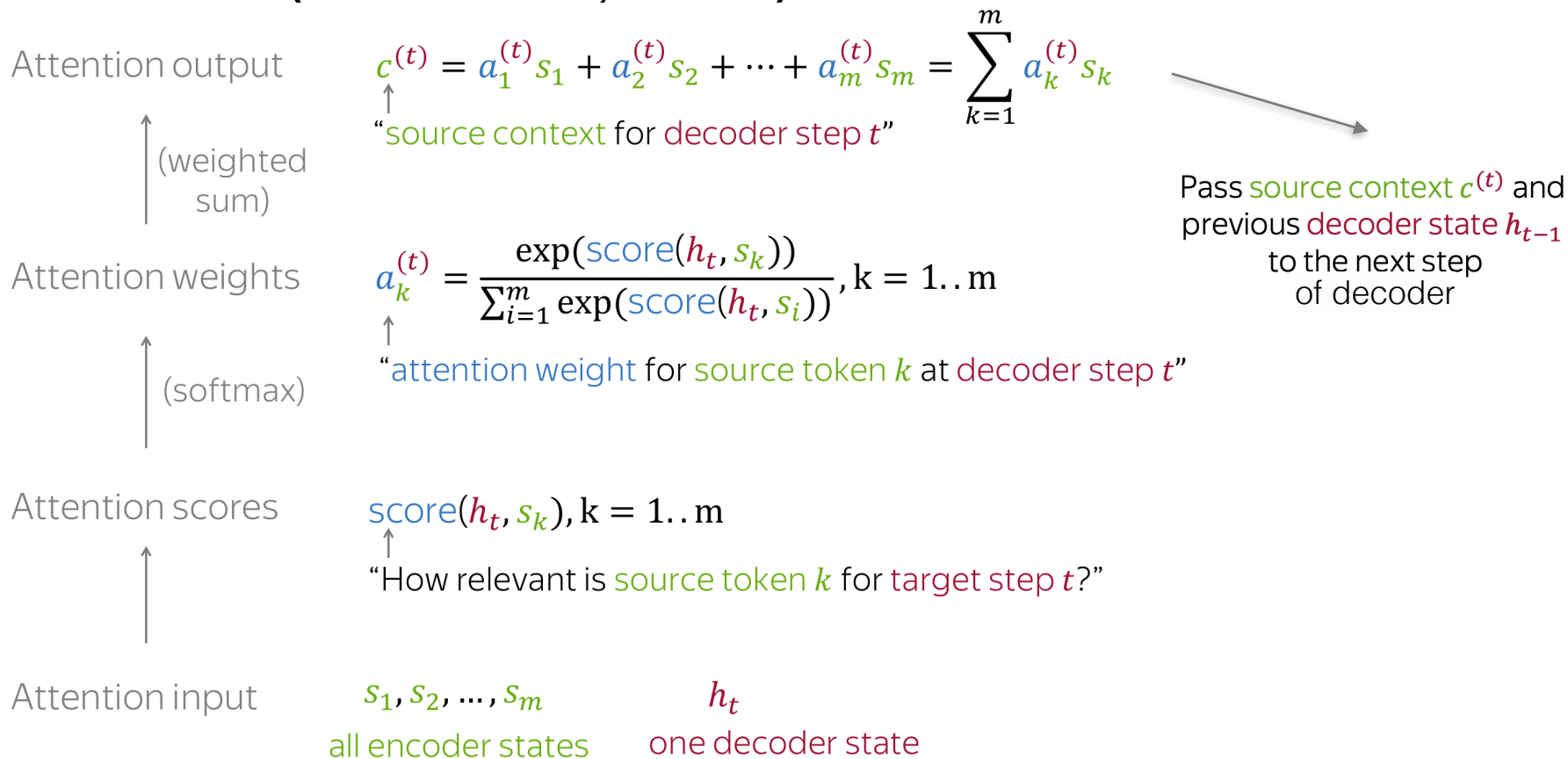
- For **encoder** fixed latent space is too small to compress all the source without loss of important information
- For **decoder** different information may be needed at different steps of generation

Idea: at different steps, let a model "*focus*" on different parts of input

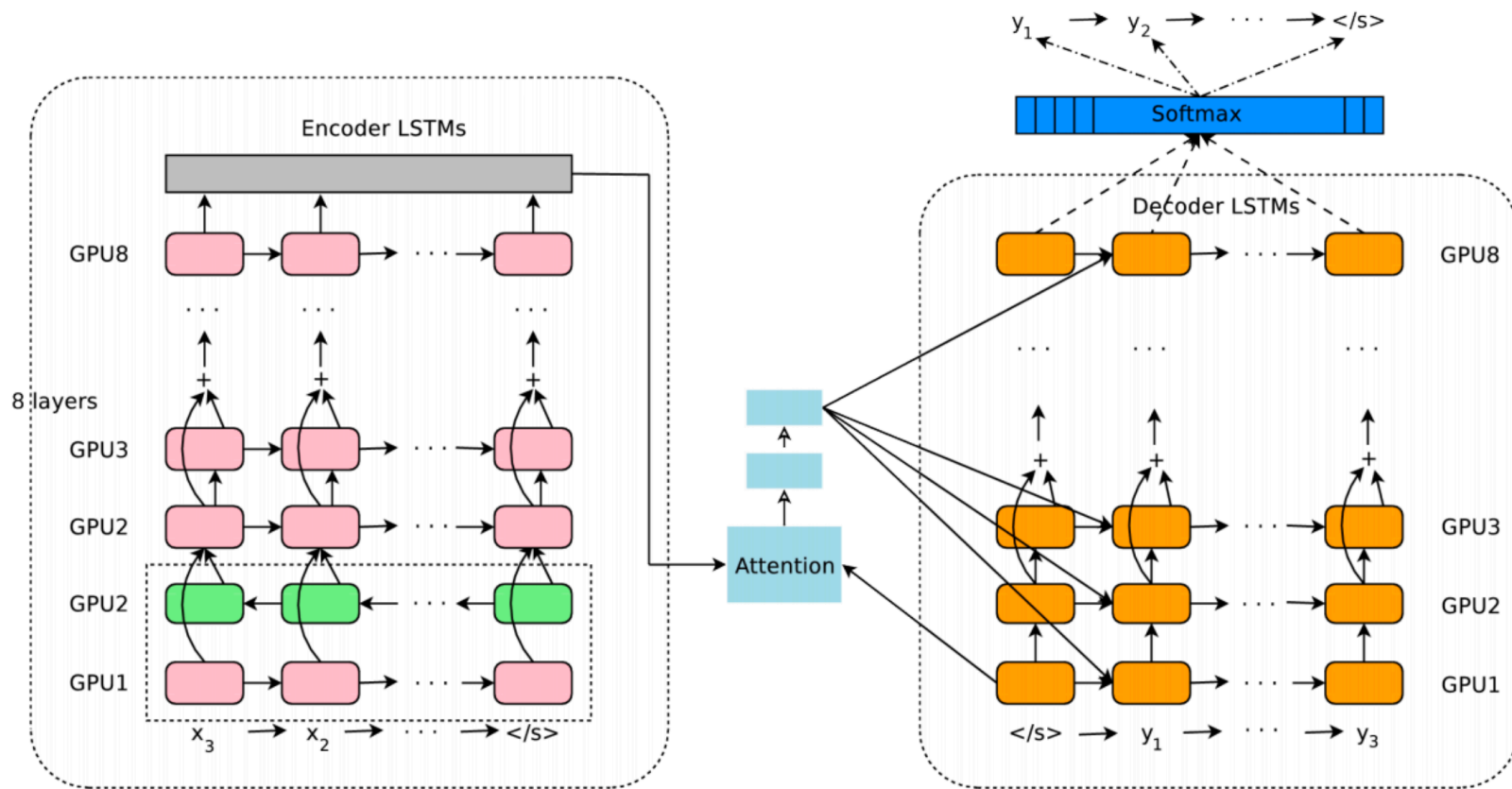
How:

- Let's keep the encoder representation of *each* token in the source text
- At each decoder step, calculate a *score* between current decoder state and each encoder state (e.g. *dot-product* or *MLP*)
- Normalize the scores and calculate the *weighted sum* of encoder states
- Use this sum as an additional input to the decoder

Attention (Bahdanau, 2014)



Google Neural Machine Translation (2016)



Transformer

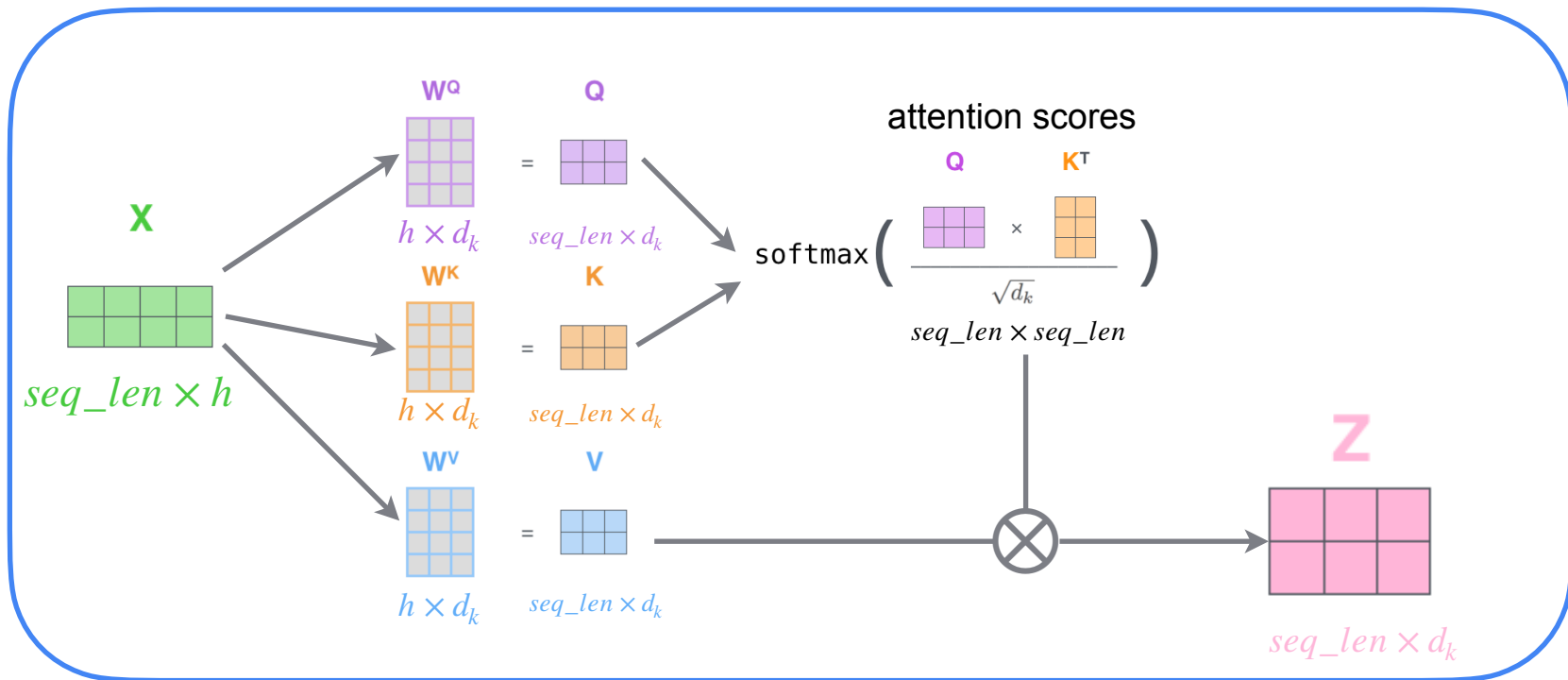
Attention is all you need (Vaswani, 2017)

Idea: replace all the recurrent NNs with blocks of attention and FFN (Feed-Forward Network) – *Transformer Blocks*

How:

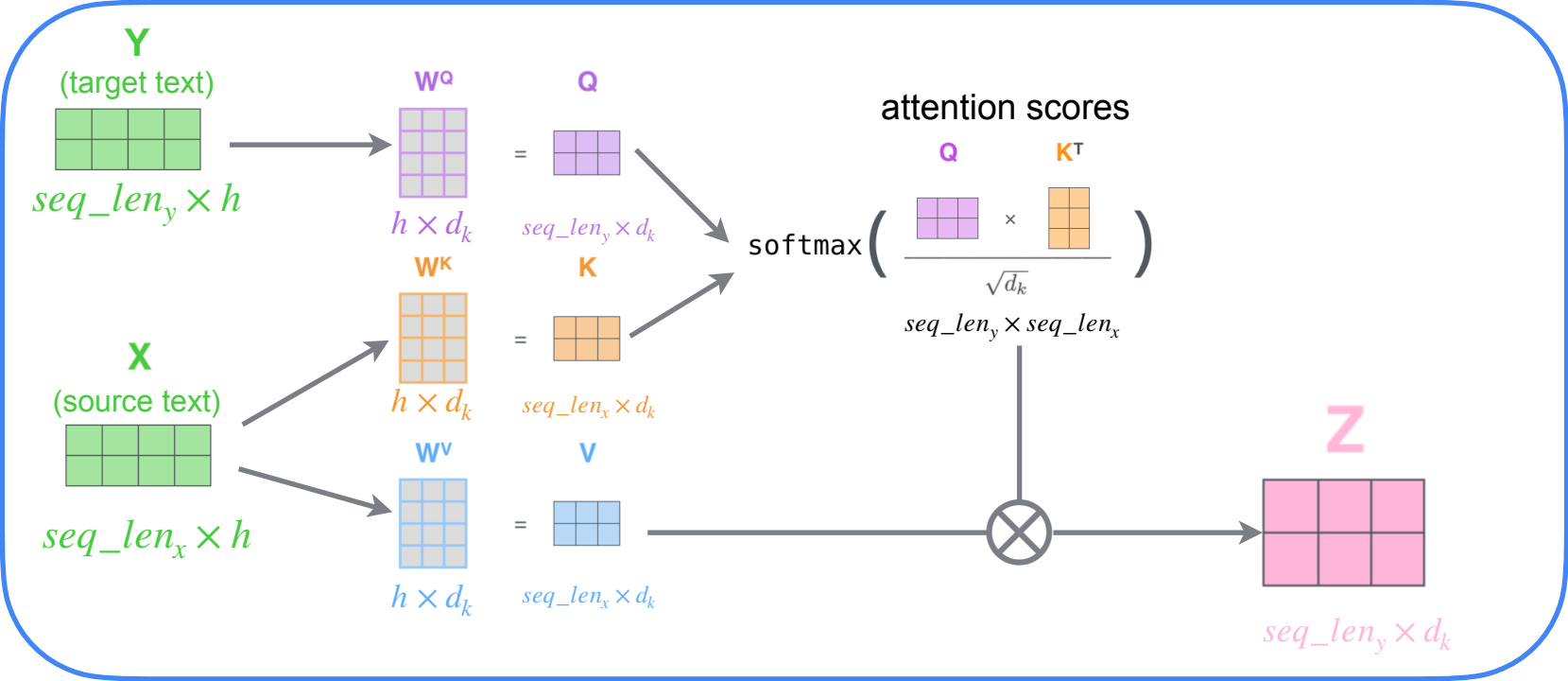
- **Encoder** – compute representation for each token using attention scores for *whole* source text (***self-attention***)
- **Decoder** – use attention scores between current hidden state and source text (***cross-attention***)
- **Decoder** – use attention scores between current hidden state and all the *previous* hidden states (***masked self-attention***)

Self-Attention Block

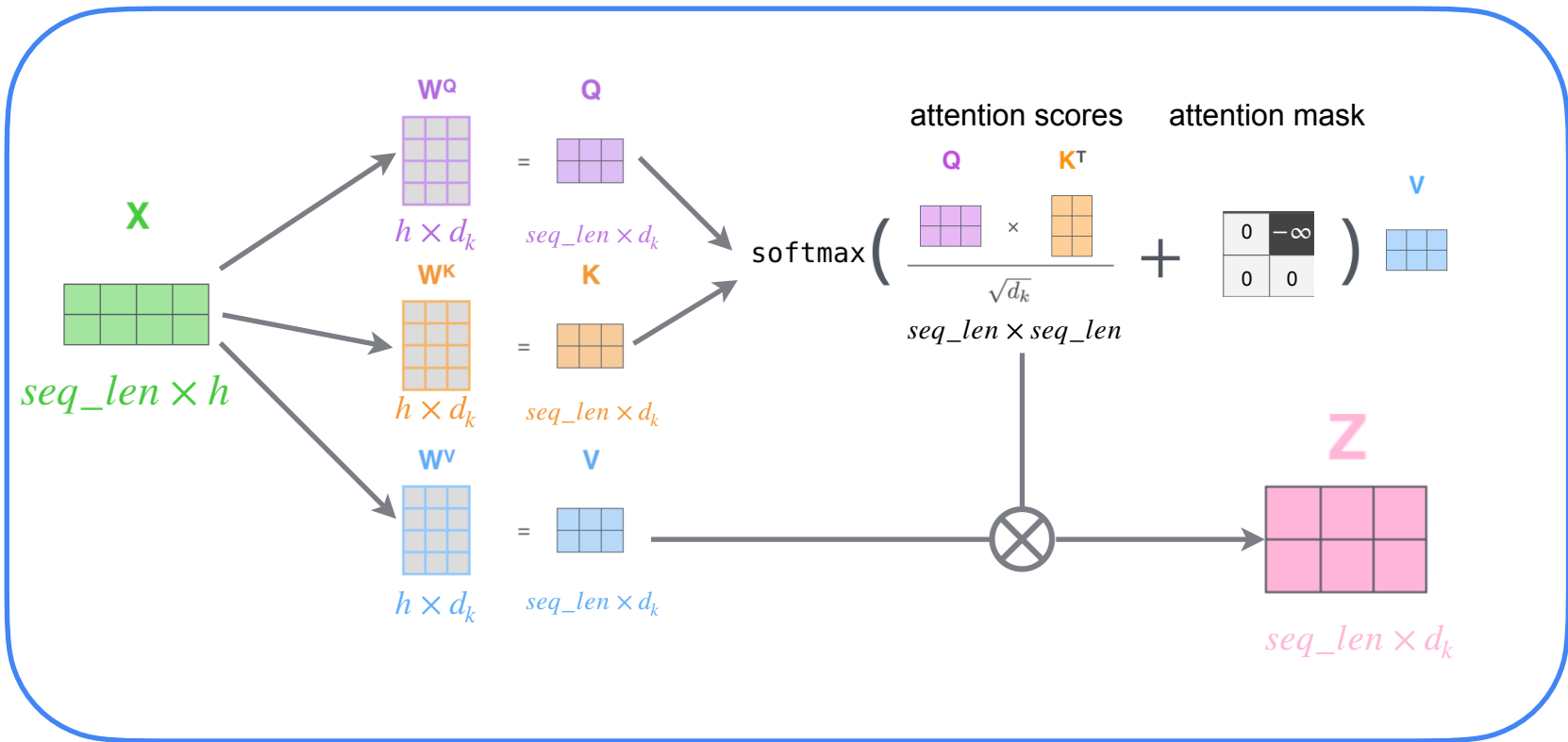


$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Cross-Attention Block

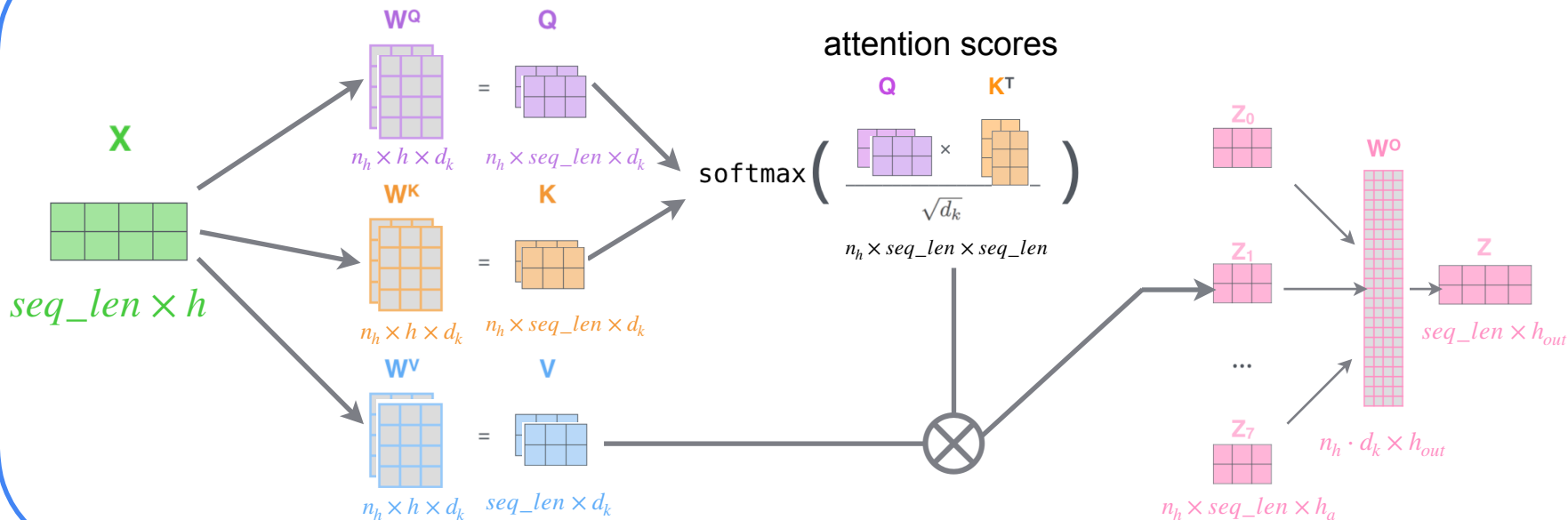


Masked Self-Attention Block



Multi-Head Attention

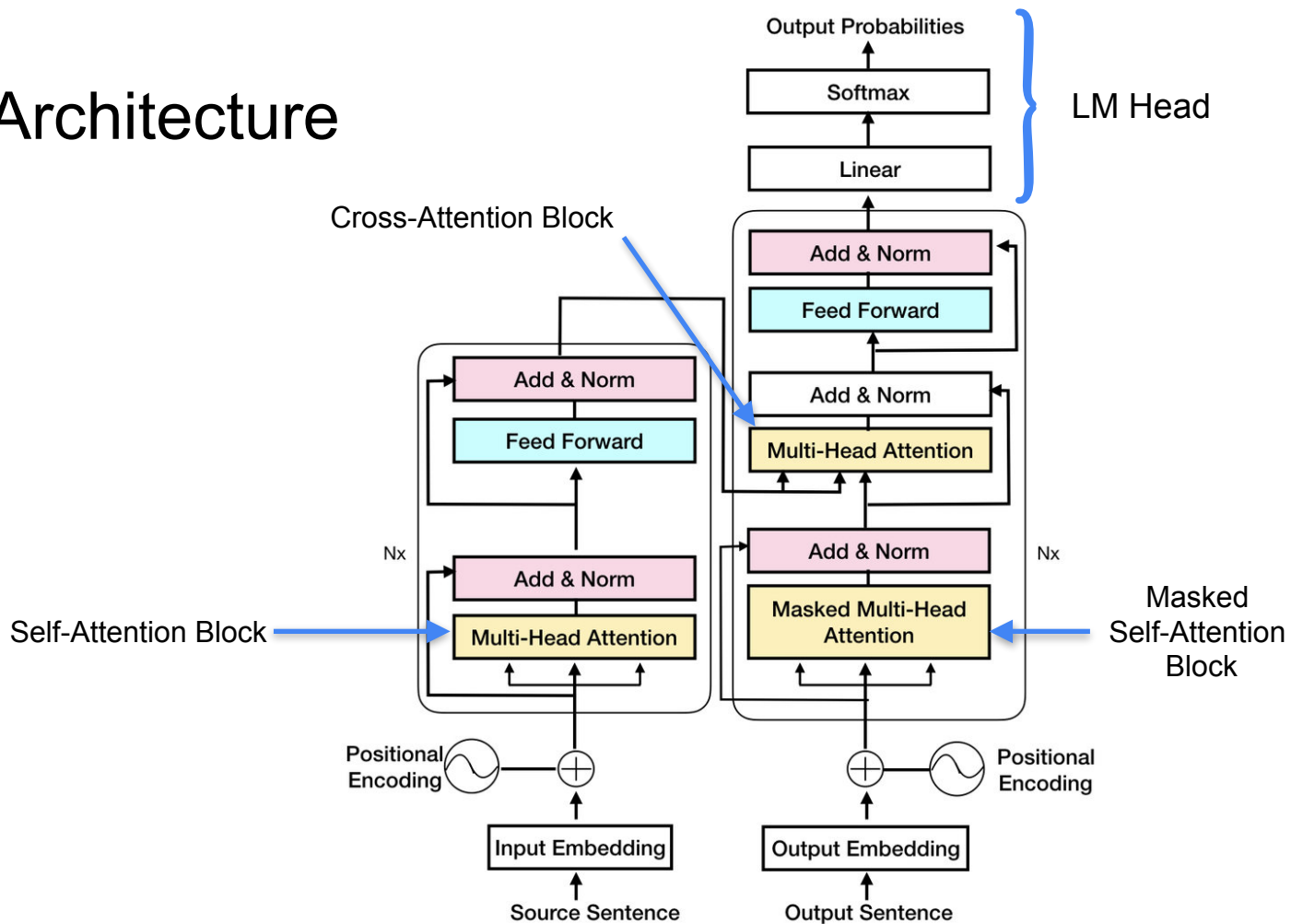
Two heads are better than one
— a proverb



$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_{n_h})W_o,$$

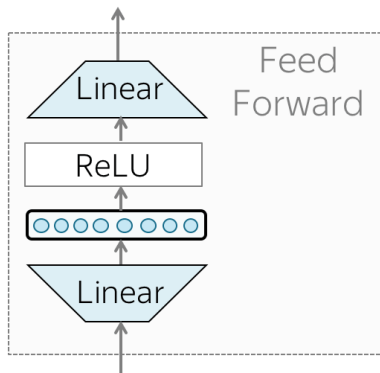
$$head_i = Attention(Q_i, K_i, W_i)$$

Transformer Architecture

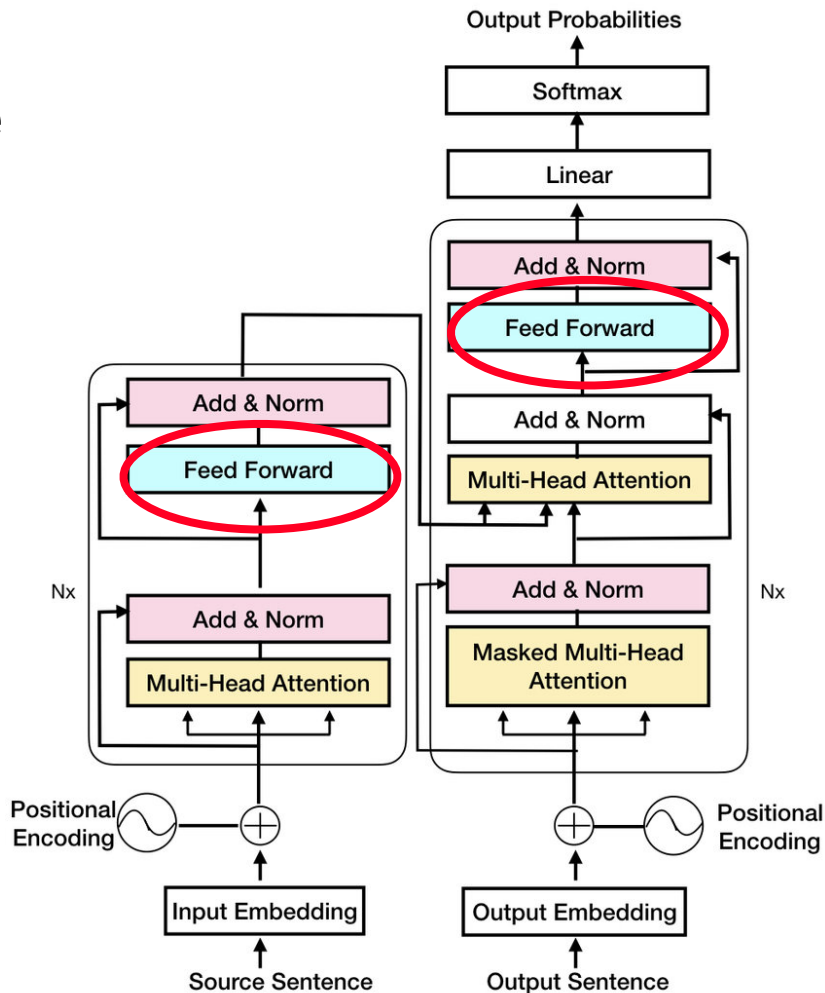


Transformer Architecture

- **Feed Forward**

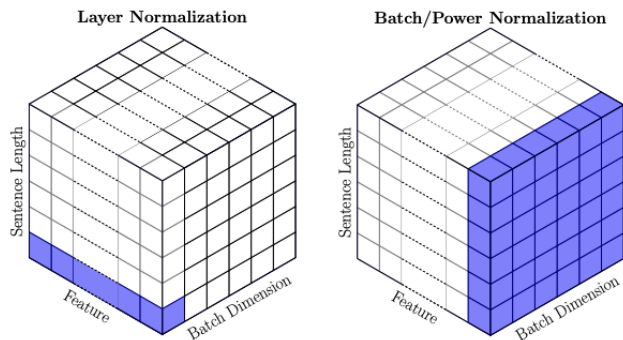


Usually something more sophisticated than ReLU, e.g. [GeLU](#) or [SiLU](#)

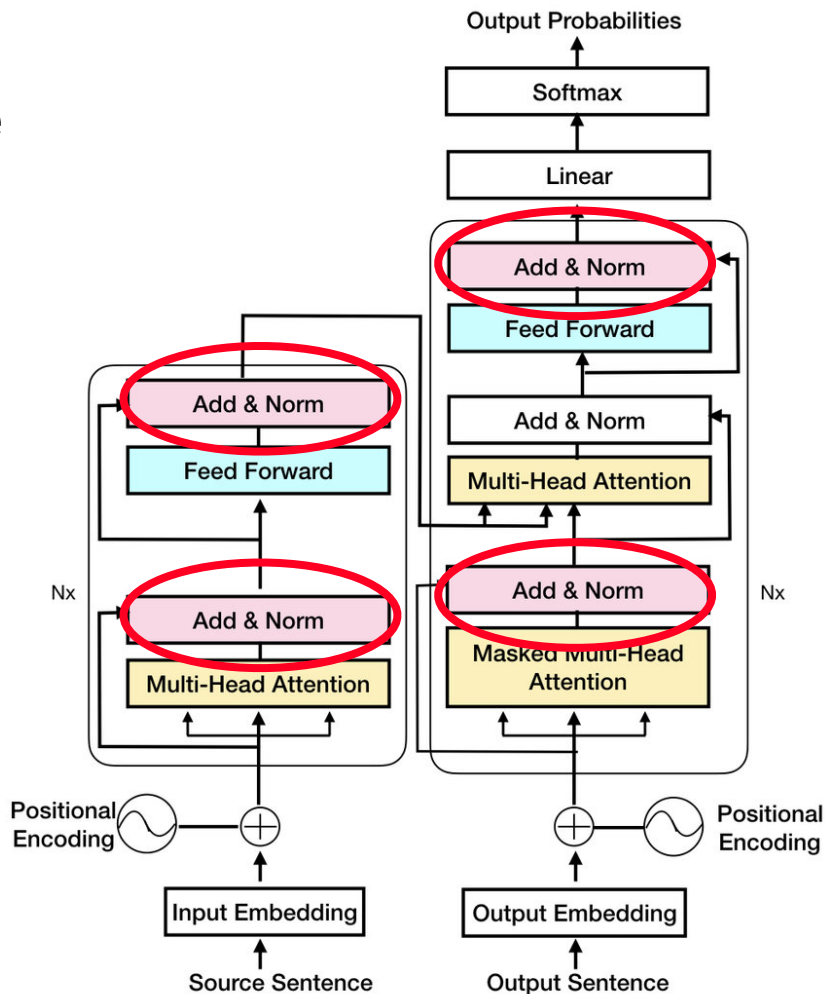


Transformer Architecture

- Feed Forward
- LayerNorm



In modern models, [RMSNorm](#) is usually used instead (has less trainable params)



Transformer Architecture

- Feed Forward

- LayerNorm

- Positional Encodings

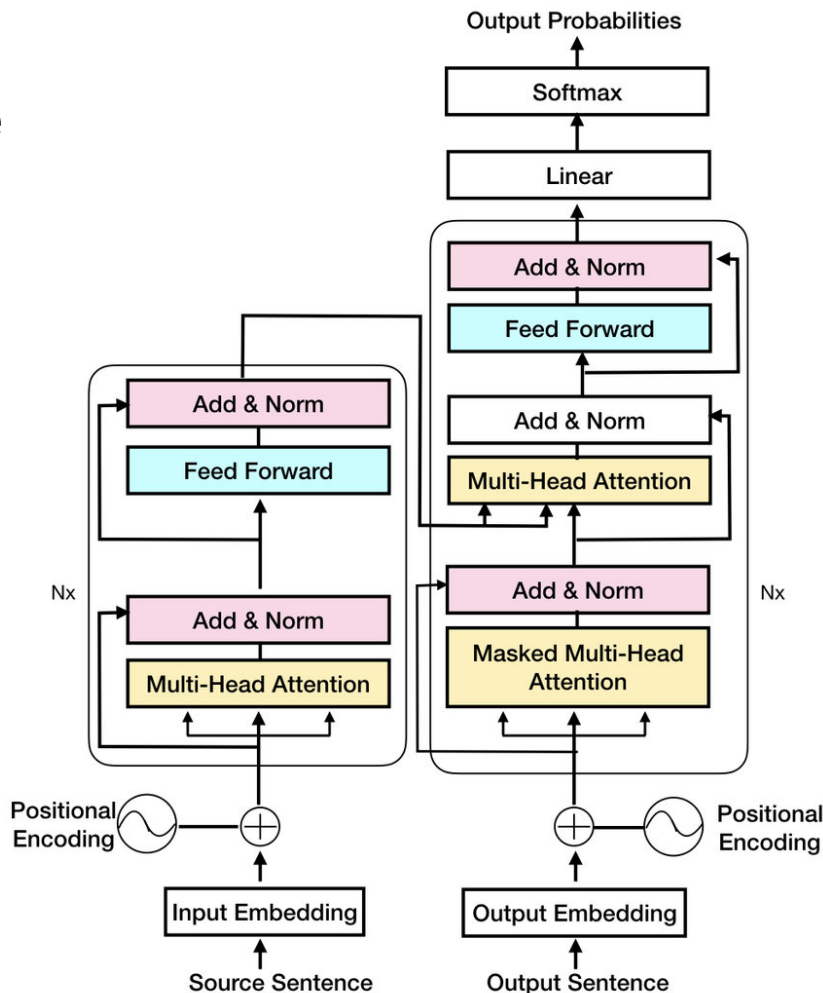
- Need to account for the order of the words
- Should output unique encoding for time-step
- Ability to generalize to long sequences

PEs suggested in Vaswani (2017):

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

In modern models more complex PEs are used, e.g. [RoPE](#)



How to measure performance?

- Perplexity

$$PPL(X) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x_i | x_{<i}) \right)$$

- Zero-shot accuracy
- Task-specific metrics

How to generate?

- *Greedy decoding* – pick token with highest probability

$$y' = \underset{y}{arg \max} \prod_{t=1}^N p(y_t | y_{<t})$$

- Sampling – sample token with probabilities obtained after LM head
- Sampling with Temperature – regulate parameter t in softmax:
- Top-K sampling
- Top-p (Nucleus) Sampling
- Beam Search